nida_supports

May 30, 2023

[ ]:
```
from datetime import date

today = date.today()
print(f'Last full compilation {today}')
```

```
Last full compilation 2023-05-30
```

[ ]:
```
from classes import *
from funcData import *
import numpy as np
import matplotlib.pyplot as plt
from math import *
import os

import matplotlib as mpl
mpl.rcParams['figure.dpi'] = 100

campagnes = ['campagne_honey']
data = get_bulk_data(read_data_from_campaigns(campagnes))


supports_data = np.zeros((len(data), 3))
supports_data[:] = np.nan
counter = 0

for test_id, test in data.items():
    with open(f'../KUBC_support_analysis/hexa/{test_id.replace("honey","hexa")}.
 ↪txt') as f:
        for _ in range(43):
            next(f)
        supports_list = []
        for l in range(40):
            line = f.readline().split(' ')
            s=0
            counter_i=1
            for it in line:
                try:
                    s=s+float(it)
```

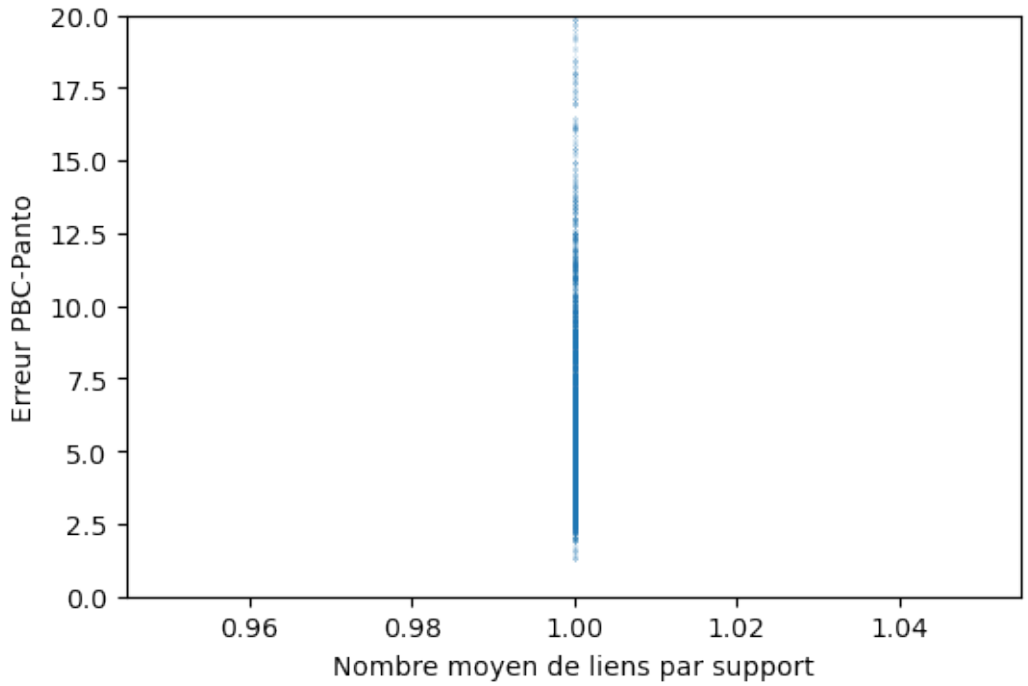
```
                    counter_i+=1
                except:
                    pass
            supports_list.append(abs(s))
    supports_data[counter, 0] = len(supports_list)/40  # Nb moyen de lien
    supports_data[counter, 1] = sum(supports_list)/(40-test.get_number_ms())␣
 ↪#incidence moyenne
    supports_data[counter, 2] = test.get_err((0,3))
    counter += 1

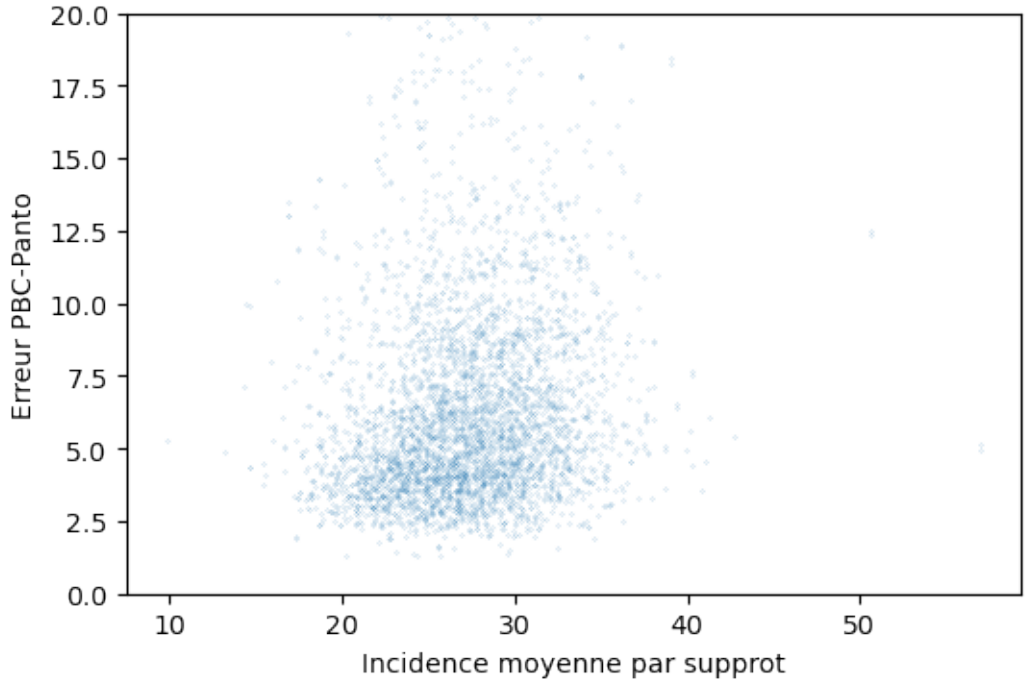
print('-  '*10)
plt.scatter(supports_data[:,0],supports_data[:,2],s=0.01)
plt.xlabel('Nombre moyen de liens par support')
plt.ylabel('Erreur PBC-Panto')
plt.ylim((0,20))

plt.figure()
plt.scatter(supports_data[:,1],supports_data[:,2],s=0.01)
plt.xlabel('Incidence moyenne par supprot')
plt.ylabel('Erreur PBC-Panto')
plt.ylim((0,20))

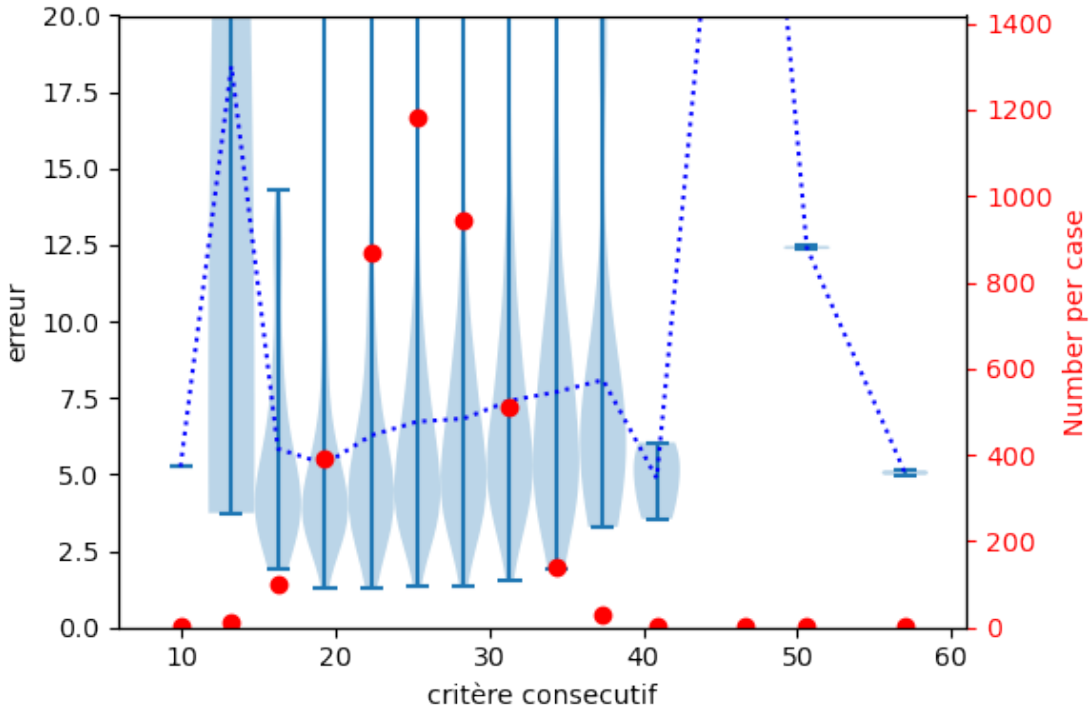
plt.figure()
plot_box2(supports_data[:,1],supports_data[:,2], step=3, ylabel=f'critère␣
 ↪consecutif', width=3,ylim=(0,20))
```

```
Lecture des données réalisée en 0.53597s
-  -  -  -  -  -  -  -  -  -
```





<Figure size 600x400 with 0 Axes>



[ ]:
```
from classes import *
from funcData import *
import numpy as np
import matplotlib.pyplot as plt
from math import *
import os

import matplotlib as mpl
mpl.rcParams['figure.dpi'] = 100

campagnes = ['campagne_honey']
data = get_bulk_data(read_data_from_campaigns(campagnes))
# data = get_tests_by_test_id(data, index=1, target='7')

supports_data = np.zeros((len(data), 3))
supports_data[:] = np.nan
counter = 0

for test_id, test in data.items():
    with open(f'../KUBC_support_analysis/hexa/{test_id.replace("honey","hexa")}.
 ↪txt') as f:
        for _ in range(84):
            next(f)
```

```
        supports_list = []
        for l in range(40):
            line = f.readline().split(' ')
            s=0
            l=[]
            counter_i=1
            for it in line:
                try:
                    s+=1/max([float(it)**3,1])
                    l.append(float(it))
                    counter_i+=1
                except:
                    pass

            try:
                supports_list.append(min(l))
            except:
                pass
    supports_data[counter, 0] = len(supports_list)/40  # Nb moyen de lien
    supports_data[counter, 1] = sum(supports_list)/(40) #incidence moyenne
    supports_data[counter, 2] = test.get_err((0,3))
    counter += 1

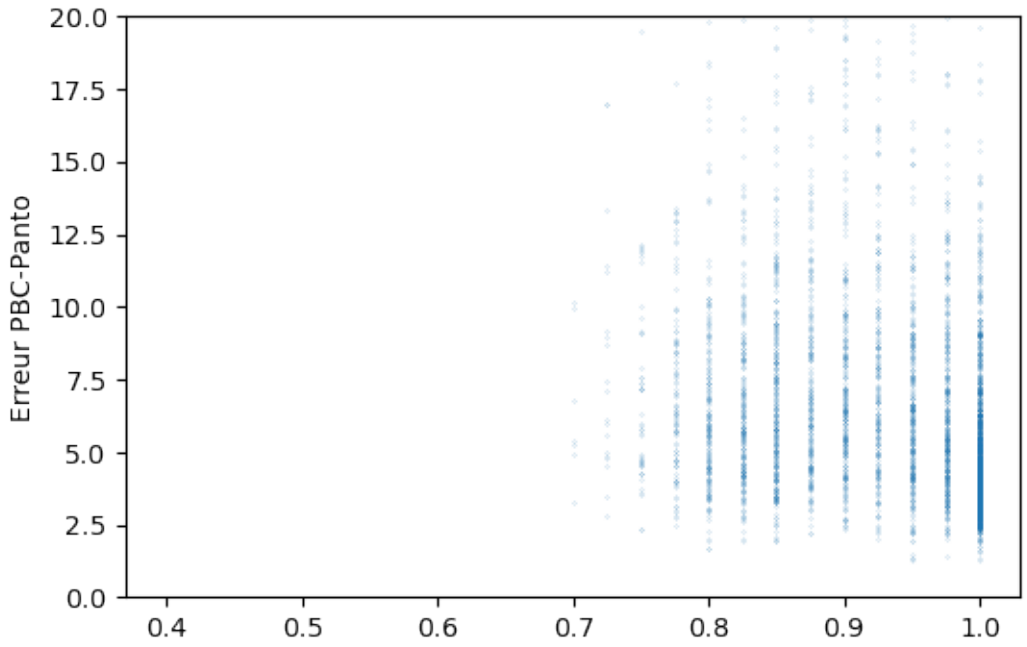
print('- '*10)
plt.scatter(supports_data[:,0],supports_data[:,2],s=0.01)
plt.ylabel('Erreur PBC-Panto')
plt.ylim((0,20))

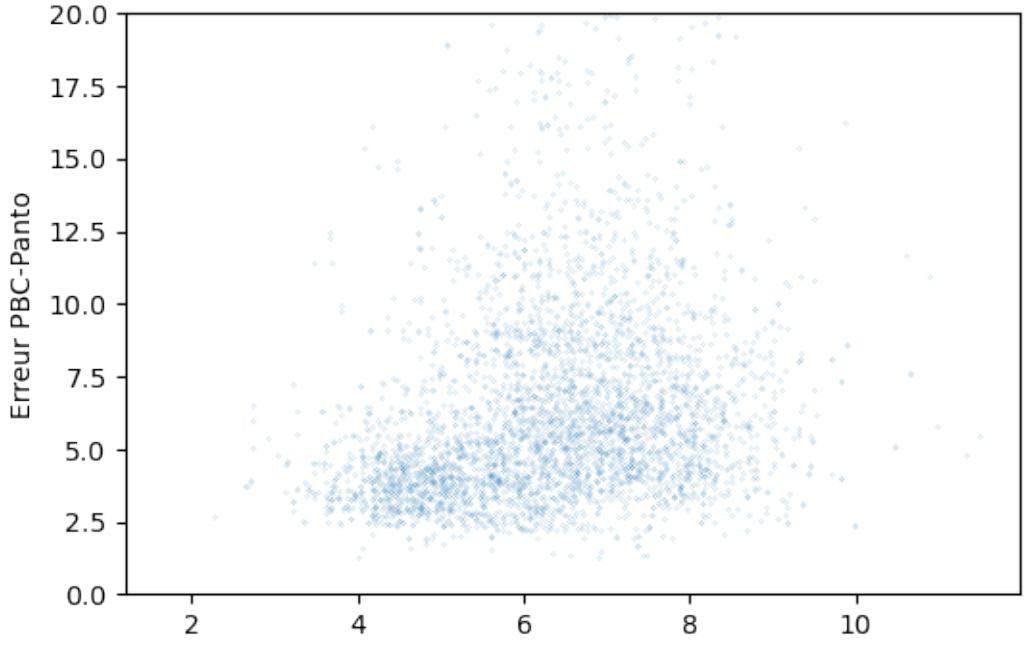
plt.figure()
plt.scatter(supports_data[:,1],supports_data[:,2],s=0.01)

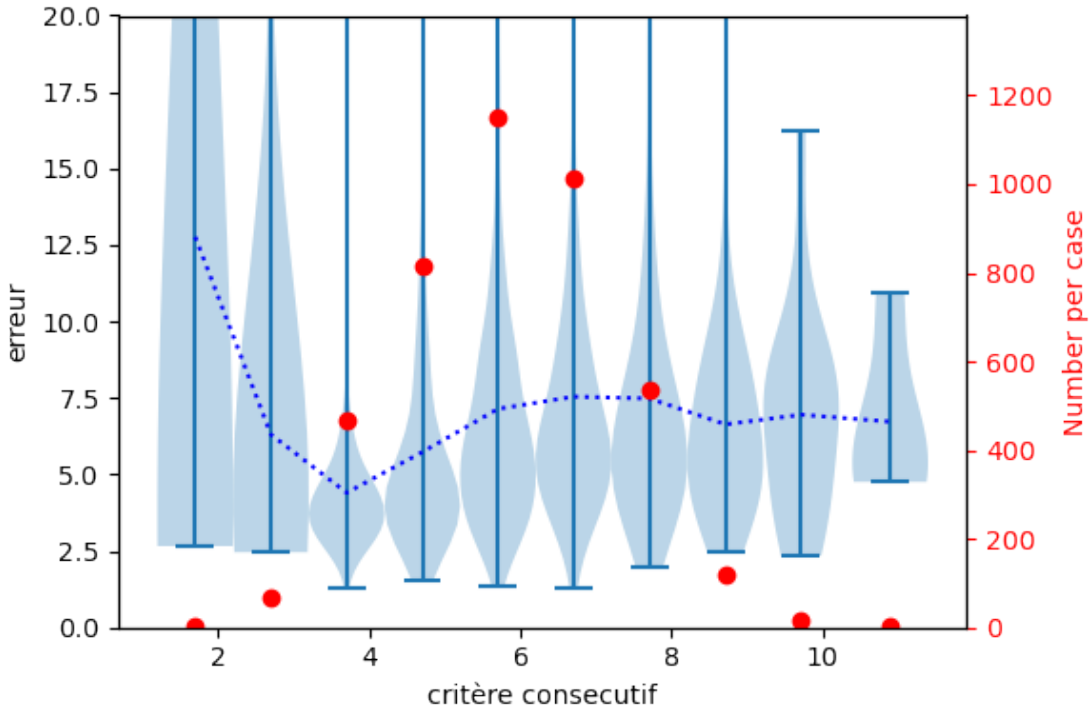
plt.ylabel('Erreur PBC-Panto')
plt.ylim((0,20))
# plt.xlim((0,0.1))

plot_box2(supports_data[:,1],supports_data[:,2], step=1.0, ylabel=f'critère␣
 ↪consecutif', width=1.0,ylim=(0.0,20))
```

```
Lecture des données réalisée en 0.44844s
- - - - - - - - - -
```


Erreur PBC-Panto
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
0.4
0.5
0.6
0.7
0.8
0.9
1.0


Erreur PBC-Panto
20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
2
4
6
8
10


20.0
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
erreur
2
4
6
8
10
critère consecutif
1200
1000
800
600
400
200
0
Number per case